

Pandas & scikit-learn Patterns You'll See This Course — Session 5

Machine Learning in Geoscience — pre-session reference · read before Session 5

Session 4 tuned models on fixed folds. Session 5 asks how forecasting actually gets verified over time, and introduces a genuinely new idea — **recursive** forecasting, where a model's own predictions become its next inputs. The patterns below are mostly about building that rolling loop and about splitting a series into successive evaluation blocks.

1. Walk-forward folds

`np.array_split(np.arange(start, n), n_folds)`

Cuts one long array of row-positions into `n_folds` roughly-equal consecutive chunks. This is how the course turns "the last 30% of the record" into 4 successive evaluation blocks, each one later than the last.

The walk-forward loop shape

For each block: train on everything strictly *before* it (minus a purge gap of `horizon` rows), predict that block, score it, then move to the next block. Unlike `TimeSeriesSplit` in Session 4 (used for tuning inside the training block), this walks forward across the *evaluation* region itself — closer to how an operational forecast is actually checked over time.

```
for k, block in enumerate(blocks, start=1):
    train_end = block[0] - horizon          # purge before this
    fold's block
    model = factory()
    model.fit(X.iloc[:train_end], y.iloc[:train_end])
    pred = model.predict(X.iloc[block])
```

2. Timestamp arithmetic

`t + pd.Timedelta(hours=h)`

Adds a fixed duration to a single Timestamp — "what time is it h hours after t?" Combined with `.loc[...]`, this looks up the true observed value at a specific future valid time, which is how the recursive roll-out is checked against reality.

```
origin = pd.Timestamp('2024-06-01 06:00')
valid_time = origin + pd.Timedelta(hours=6)
truth = hourly['wind_speed_ms'].loc[valid_time]
```

3. Recursive roll-out: feeding predictions back in

The core idea

A **direct** model (Sessions 2-4) is trained once per horizon and predicts that horizon straight from real observations. A **recursive** model is trained once, for one step ahead ($T \rightarrow T+1$), then run 24 times in a row: each prediction becomes part of the "observed" history fed into the next step's features. This is the one genuinely new mechanism this session — everything else is bookkeeping around it.

```
history = list(known_past_values)
for step in range(24):
    features = build_features_from(history)  # uses predictions,
    not just real data
    next_pred = model.predict(features)
    history.append(next_pred)                # feed it back in
```

`recursive_path[:, h - 1]`

The roll-out is stored as a 2-D array — one row per forecast origin, one column per lead time. Column `h-1` holds every origin's prediction for "h hours ahead," used here to plot how recursive error grows with lead time.

4. Cyclical calendar features

`np.sin(2*np.pi*hour/24)`, `np.cos(2*np.pi*hour/24)`

Encodes "hour of day" as a point on a circle instead of a plain number, so that 23:00 and 00:00 end up close together (as they should be) rather than maximally far apart. Always used as a *pair* — sine alone can't distinguish, say, 6:00 from 18:00.

5. Building an array from a loop over timestamps

`np.array([series.loc[t] for t in origins])`

A list comprehension that looks up one value per origin timestamp, then converts the whole list to a NumPy array in one step — used to pull the true future values that the recursive roll-out is compared against.

Before Session 5 — a 10-minute self-check

- Take a toy array of 20 numbers and split it into 4 consecutive chunks with `np.array_split`.
- Write a tiny loop that predicts one step, appends the prediction to a list, and uses the updated list to "predict" the next step — the skeleton of a recursive roll-out, no real model needed.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 5 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.